

Objetivos da Aula:

- transformar arquivo solto em modulo, com export e import.
- organizar as rotas da eleicao com o Router do Express.
- instalar um motor de views e devolver paginas HTML de arquivo.
- injetar variaveis na pagina e reaproveitar cabecalho e rodape.

De onde viemos e para onde vamos

Na Aula 2 nos fizemos:

revisamos array e objeto, a forma de guardar os dados da eleicao.

instalamos express e nodemon.

subimos o servidor e lemos parametro pela URL, com req.params.

leamos parametro pela query string, com req.query.

Hoje o assunto e outro:

o index.js esta com toda rota dentro de um arquivo so. Vamos separar.

import e export deixam de ser vistos de longe e viram ferramenta do dia a dia.

candidatos e votos ganham arquivo proprio, sem banco de dados ainda.

e o servidor passa a devolver pagina de verdade, escrita em arquivo.

Projeto modelo - eleição fictícia

Iniciar projeto do zero:

quais os passos das aulas anteriores?

npm o quê?

quais dependências?

```

eleicao/
├── index.js           # ponto de entrada: cria o app, configura o EJS e liga as rotas
├── package.json      # arquivo de configuração e dependências do projeto
├── src/
│   ├── dados/        # dados em memória, no lugar de um banco (ainda)
│   │   ├── index.js  # reexporta candidatos e votos (ponto único de importação)
│   │   ├── candidatos.js # export const candidatos = [...]
│   │   └── votos.js  # export const votos = [...]
│   ├── dominio/      # as classes: as regras de negócio da eleição
│   │   ├── Candidato.js # valida nome (min. 3 letras) e número (2 dígitos)
│   │   ├── Usuario.js  # login/senha + método autenticar()
│   │   ├── Voto.js     # exige um candidatoId
│   │   └── Eleicao.js   # junta tudo: cadastrar(), registrarVoto(), apurar()
│   ├── routes/       # as rotas HTTP, cada arquivo é um Router do Express
│   │   ├── paginaInicial.js # GET / -> renderiza a view home.ejs
│   │   ├── candidatoRoutes.js # GET /candidatos -> HTML fixo
│   │   │   # GET /candidatos/:numero -> JSON de um candidato
│   │   ├── paginaRoutes.js  # GET /pagina/candidatos -> lista os candidatos em HTML
│   │   └── votoRoutes.js    # (vazio - será construído na aula)
│   └── views/
│       └── home.ejs        # página inicial, recebe { titulo, candidatos }
└── ...

```

O index.js de onde partimos

```
import express from 'express'

const app = express()

// funcao callback

app.get('/busca', (req, res) => {
  console.log(req.query.nome)
  res.send('ola mundo')
})

app.get('/busca/:xx', (req, res) => {
  const numero = req.params.xx
  console.log(numero)
  const mensagem = `0 numero requisitado e: ${numero}`
  res.send(mensagem)
})

app.listen(3000, () => {
  console.log('Servidor rodando em http://127.0.0.1:3000')
})
```

Uma analogia antes de qualquer definicao

Pense numa escola com uma sala so, onde ficam todas as turmas, todo o material e a secretaria juntos. Funciona no comeco, mas vira bagunca assim que a escola cresce.

Separar em salas resolve isso.

Cada sala guarda um assunto. Na porta de cada sala existe uma placa dizendo o que tem la dentro: isso e o export.

Ir buscar o que precisa em outra sala e o import.

Voce nao entra em qualquer sala e pega qualquer coisa: so pega o que a placa da porta anunciou.

A secretaria continua existindo, so que agora ela so encaminha: nao guarda mais nada sozinha. No nosso projeto, essa secretaria e o index.js.

export nomeado e export default

export nomeado

um arquivo pode ter varios.

o import usa chaves e repete o mesmo nome.

```
export const candidatos = [ ... ]
```

```
import { candidatos } from './candidatos.js'
```

export default

so um por arquivo.

o import escolhe o nome que quiser, sem chaves.

```
export default candidatos
```

```
import candidatos from './candidatos.js'
```

Neste curso usamos nomeado para dado e default para o router de cada arquivo.

src/dados/candidatos.js

```
export const candidatos = [  
  {  
    id: 1,  
    nome: 'Rick Sanchez',  
    numeroUrna: '10',  
    imageUrl: 'https://rickandmortyapi.com/api/character/avatar/1.jpeg',  
    cargoId: 1,  
    ativo: true,  
  },  
  {  
    id: 2,  
    nome: 'Morty smith',  
    numeroUrna: '20',  
    imageUrl: 'https://rickandmortyapi.com/api/character/avatar/2.jpeg',  
    cargoId: 1,  
    ativo: true,  
  },  
  {  
    id: 3,  
    nome: 'Summer smith',  
    numeroUrna: '30',  
    imageUrl: 'https://rickandmortyapi.com/api/character/avatar/3.jpeg',  
    cargoId: 1,  
    ativo: true,  
  }  
];
```

src/dados/votos.js

```
// Cada voto guarda so o id do candidato que recebeu.
```

```
export const votos = [  
  { id: 1, tipo: 'nominal', candidatoId: 1 },  
  { id: 2, tipo: 'nominal', candidatoId: 1, },  
  { id: 3, tipo: 'nominal', candidatoId: 2 },  
  { id: 4, tipo: 'branco', candidatoId: null},  
  { id: 5, tipo: 'nulo', candidatoId: null },  
];
```


import: pegando o que outro arquivo exportou

```
import { candidatos } from './src/dados/candidatos.js'  
import { votos } from './src/dados/votos.js'
```

```
console.log(candidatos.length)    // 3  
console.log(votos[0].candidatoId) // 1
```

```
// O caminho começa com ./ porque é um arquivo do projeto,  
// e termina com .js: em import e export, a extensão  
// não é opcional.
```

Erro classico: import sem a extensao .js

```
import { candidatos } from './src/dados/candidatos'
```

// Saida real do terminal:

```
// Error [ERR_MODULE_NOT_FOUND]: Cannot find module  
// '../src/dados/candidatos' imported from index.js
```

```
// Correcao: escreva o caminho inteiro, com .js no final.  
import { candidatos } from './src/dados/candidatos.js'
```

Erro classico: misturar nomeado com default

```
// src/dados/candidatos.js exportou default:  
export default candidatos
```

```
// index.js tentou importar com chaves:  
import { candidatos } from './src/dados/candidatos.js'
```

```
// Saida real do terminal:
```

```
// SyntaxError: The requested module  
// './src/dados/candidatos.js' does not provide an  
// export named 'candidatos'
```

```
// Export e import precisam combinar: os dois nomeados,  
// ou os dois default.
```

Router: uma sala para cada assunto de rota

Ate aqui, toda rota morava dentro de index.js. Funciona com duas rotas. Com o projeto inteiro, vira a sala unica de novo.

O Router do Express e a sala das rotas de um assunto.

Um arquivo declara suas proprias rotas com Router, e exporta o resultado pronto.

O index.js so registra a sala com app.use.

Ele vira a secretaria: encaminha, nao implementa.

src/routes/candidatoRoutes.js

```
import { Router } from 'express'
import { candidatos } from '../dados/candidatos.js'
import { Candidato } from '../dominio/Candidato.js'

const ana = new Candidato(1, 'Ana Souza', '10')
console.log(ana.nome) // Ana Souza
try {
  const invalido = new Candidato(2, 'A1', '20')
  console.log('cadastrado:', invalido)
} catch (erro) {
  console.log('falhou:', erro.message)
}

const router = Router()
router.get('/candidatos', (req, res) => {
  res.send(`
    <html>
      <body>
        <h1>Eleicao</h1>
        <p>Bem vindo</p>
      </body>
    </html>
  `)
})

router.get('/candidatos/:numero', (req, res) => {
  const numero = req.params.numero
  const encontrado = candidatos.find((c) => c.numeroUrna === numero)
  if(!encontrado){
    res.json({"mensagem": "não encontrado"})
  }
  res.json(encontrado)
})

export default router
```

Por que separar rotas em arquivos

- um arquivo, um assunto: quem procura a rota de voto abre votoRoutes.js.
- index.js vira so a recepcao: monta tudo, nao implementa nada.
- crescer o projeto nao significa um arquivo gigante: significa mais uma sala.
- duas pessoas em arquivos diferentes nao se atropelam.

O limite do res.send

res.send devolve texto. Uma pagina HTML tambem e texto, entao da para montar a pagina inteira dentro de uma template string e mandar.

Funciona, e nao escala.

A pagina fica presa dentro do arquivo de rota, misturada com a logica. Cada pagina nova aumenta o arquivo. E quem mexe em HTML precisa abrir codigo JavaScript para achar a tag.

A solucao e separar o HTML em arquivo proprio.

A rota decide os dados, o arquivo de pagina decide a aparencia. Quem faz essa ponte e o motor de views.

Instalando o motor de views: EJS

```
// No terminal, dentro da pasta do projeto:
```

```
npm install ejs
```

```
// Saida real:
```

```
// added 1 package, and audited 70 packages in 2s
```

```
// Conferindo no package.json:
```

```
// "dependencies": {  
//   "ejs": "^6.0.1",  
//   "express": "^4.22.2"  
// }
```

```
// EJS quer dizer Embedded JavaScript:
```

```
// HTML normal com pedacinhos de JavaScript embutidos.
```


Configurando o Express para usar views no index.js

```
import express from 'express'
import candidatoRoutes from './src/routes/candidatoRoutes.js'
import paginaRoutes from './src/routes/paginaRoutes.js'
import paginaInicial from './src/routes/paginaInicial.js'
import path from 'node:path'

const app = express()
// 1. qual motor de views usar
app.set('view engine', 'ejs')
// 2. em qual pasta estao os arquivos de pagina
app.set('views', path.resolve('src/views'))
// 3. pasta de arquivos publicos: css, imagem, js do navegador
app.use(express.static('public'))

app.use(candidatoRoutes)
app.use(paginaRoutes)
app.use(paginaInicial)
app.listen(3000, () => {
  console.log('Servidor rodando em http://127.0.0.1:3000')
})
```

views/home.ejs: a primeira pagina em arquivo

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="utf-8">
  <title><%= titulo %></title>
  <link rel="stylesheet" href="/css/estilo.css">
</head>
<body>
  <h1><%= titulo %></h1>

  <p>A eleicao tem <%= candidatos.length %> candidatos.</p>
  <a href="/candidatos">ver os candidatos</a>
</body>
</html>
```

res.render: a rota entrega os dados para a pagina

```
import { Router } from 'express'
import { candidatos } from '../dados/candidatos.js'

const router = Router()

router.get('/', (req, res) => {
  res.render('home', { titulo: 'Eleicao', candidatos })
})

export default router

// 1o argumento: o nome do arquivo em views, sem o .ejs
// 2o argumento: um objeto com tudo que a pagina vai usar.
// Cada chave desse objeto vira uma variavel dentro da view.
```

As tres tags do EJS

<%= valor %>

imprime o valor na pagina, escapando o HTML.
e a que voce vai usar quase sempre.

<%- valor %>

imprime sem escapar: interpreta o HTML.
usado para incluir outro arquivo.

<% comando %>

executa JavaScript e nao imprime nada.
e onde entram if e for.

Saida real com texto = "Ana":

<%= texto %> imprime Ana

<%- texto %> imprime Ana em negrito

views/candidatos.ejs: laco dentro da pagina

```
<h1><%= titulo %></h1>
```

```
<ul>
```

```
  <% for (const candidato of candidatos) { %>
```

```
    <li><%= candidato.numero %> - <%= candidato.nome %></li>
```

```
  <% } %>
```

```
</ul>
```

```
// A chave que abre o for fica dentro de <% %>,
```

```
// a chave que fecha tambem: <% } %>
```

```
// O HTML no meio se repete uma vez por candidato.
```

Saida real: o HTML que chega no navegador

```
<ul>  
  <li>10 - Ana Souza</li>  
  <li>20 - Bruno Lima</li>  
  <li>30 - Carla Dias</li>  
</ul>
```

O navegador nunca ve o EJS.

Ele recebe HTML puro, ja com os dados dentro.

Quem monta isso e o servidor, antes de responder.

Partials: cabecalho e rodape reaproveitados

```
// views/partials/cabecalho.ejs
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="utf-8">
  <title><%= titulo %></title>
  <link rel="stylesheet" href="/css/estilo.css">
</head>
<body>
  <h1><%= titulo %></h1>

// views/partials/rodape.ejs
</body>
</html>

// E agora a pagina fica so com o que e dela:
<%- include('partials/cabecalho') %>
  <p>A eleicao tem <%= candidatos.length %> candidatos.</p>
<%- include('partials/rodape') %>
```

public/css/estilo.css e o express.static

```
/* public/css/estilo.css */

body { font-family: Arial, sans-serif; margin: 40px; }
h1    { color: #2A8F7D; }

// No index.js:
app.use(express.static('public'))

// Na view, o caminho começa na RAIZ do public:
// <link rel="stylesheet" href="/css/estilo.css">

// e nao href="/public/css/estilo.css".
// A pasta public some do endereco: ela vira a raiz.
```


Erro classico: view nao encontrada

```
res.render('naoexiste', {})
```

```
// Saida real do navegador, com status 500:
```

```
// Error: Failed to lookup view "naoexiste"  
//       in views directory "/projeto/views"
```

```
// Causas mais comuns:
```

```
// 1. o arquivo nao existe, ou o nome tem erro de digitacao  
// 2. faltou o app.set('views', ...)  
// 3. escreveu a extensao: render("home.ejs") em vez de "home"
```

Erro classico: variavel que a view usa e a rota nao mandou

```
// A rota mandou so o titulo:  
res.render('home', { titulo: 'Eleicao' })  
  
// Mas a view usa candidatos:  
// <p><%= candidatos.length %></p>  
  
// Saida real:  
  
// ReferenceError: candidatos is not defined  
  
// A view so enxerga o que veio no objeto do render.  
// Nao existe variavel global entre rota e pagina.
```

Maos a obra: a estrutura de hoje

Codigo do servidor:

index.js
src/dados/candidatos.js
src/dados/votos.js
src/routes/candidatoRoutes.js
src/routes/paginaRoutes.js

Paginas e arquivos publicos:

views/home.ejs
views/candidatos.ejs
views/partials/cabecalho.ejs
views/partials/rodape.ejs
public/css/estilo.css

index.js completo, no fim da aula

```
import express from 'express'
import path from 'node:path'
import candidatoRoutes from './src/routes/candidatoRoutes.js'
import paginaRoutes from './src/routes/paginaRoutes.js'

const app = express()

app.set('view engine', 'ejs')
app.set('views', path.resolve('views'))
app.use(express.static('public'))

app.use(candidatoRoutes)
app.use(paginaRoutes)

app.listen(3000, () => {
  console.log('Servidor rodando em http://127.0.0.1:3000')
})
```

Para praticar

- Crie views/votos.ejs listando os votos, e a rota /votos que renderiza essa pagina.
- Faça a pagina de candidatos mostrar, ao lado de cada nome, quantos votos ele recebeu.
- Use `<% if (candidatos.length === 0) { %>` para mostrar "nenhum candidato" quando a lista estiver vazia.
- Provoque o erro de proposito: peca uma variavel na view sem mandar no render, e leia a mensagem.
- Acrescente um menu no partial do cabecalho, com link para todas as paginas, e veja aparecer em todas de uma vez.

Referencias

Express.js. Routing e Using template engines with Express.

<https://expressjs.com/en/guide/routing.html>

EJS. Documentacao oficial das tags.

<https://ejs.co/>

MDN Web Docs. import, export, Modulos JavaScript.

<https://developer.mozilla.org/pt-BR/>

PEREIRA, Caio Ribeiro. Livro de NodeJS. Casa do Codigo, 2013. Atualizado em 09/2021.